产品设计文档：本地知识助手（从 RAG 到 Agentic RAG 的演进实践）

- 1. 项目概述（TL;DR）
- 2. 目标用户与核心场景
- 3. 问题定义：为什么这个项目成立
- 4. 现有方案对比与技术路线判断
- 5. V1：基础 RAG 产品与系统设计
- 6. V1 验证体系与 Badcase 闭环
- 7. V2：Agentic RAG 产品与系统设计
- 8. V2 验证体系与 Badcase 闭环
- 9. V1 vs V2 对比
- 10. 关键产品决策与 Trade-off
- 11. 效果评估与用户价值
- 12. 未来扩展
- 13. 我的角色（AI PM）

1. 项目概述（TL;DR）

1.1 一句话总结

本项目是一个面向个人知识管理和项目复盘场景的本地知识助手：V1 用基础 RAG 解决“资料找得回、答案有依据”，V2 在 RAG 之上加入 Router、search_docs Tool、Memory 和 Agent Core，让系统从固定检索问答链路升级为可控的轻量任务型知识助手。

1.2 项目定位

当前定位是**个人知识工作台 MVP**，不是成熟商业化产品，也不是企业级知识库系统。

它重点验证三件事：

1. 个人本地资料能否通过语义检索被找回。
2. AI 回答能否绑定来源，支持用户回到原文复核。
3. 在基础 RAG 可信后，Agent 控制层能否让系统支持任务判断、工具调用和短期连续追问。

> **当前表达边界**
> 这不是“我做了一个商业级 AI Agent 产品”，而是“我基于真实个人知识工作流，跑通了本地 RAG 到 Agentic RAG 的 MVP，并能说明用户、问题、方案取舍、验证体系、badcase 和后续产品化前提”。

1.3 问题 / V1 / V2 / 验证结果 四点摘要

维度	摘要
问题	个人笔记、项目资料、作品集素材和本地文档分散在 Obsidian 与本地文件中；关键词搜索难覆盖同义表达和跨文件线索；直接问 LLM 又缺少本地资料和来源依据。
V1	建立本地基础 RAG：文档读取与转换、chunk 切分、本地 embedding、Chroma 向量检索、来源片段召回、基于来源回答和 FastAPI Web GUI。
V2	在 V1 之上加入 Agent 控制层：Router 判断 direct / retrieve / clarify，RAG 封装为 `search_docs` Tool，Memory 保存短期上下文，Agent Core 负责任务调度和结果生成。
验证结果	已完成 163 个文件、390 个 chunks 的本地索引验证；10 条真实任务问题中 9 条命中，1 条基本命中；回答可用性人工判断约 8/10。

1.4 当前边界：个人知识工作台 MVP，不包装成商业化产品

当前项目不声称具备以下能力：

- 不声称已经是企业级多用户权限系统。
- 不声称经过线上 A/B 或大规模用户验证。
- 不声称 Agent 已具备复杂多步骤自主执行能力。
- 不声称 fine-tuning、Planner、多 Agent、自动写回都已经落地。
- 不默认把私人资料上传到云端知识库。

当前更准确的说法是：

> 在个人真实知识工作流中，项目已经验证本地语义检索、来源问答、GUI 使用、轻量 Agent 路由、短期 Memory 和安全边界的 MVP 可行性。

2. 目标用户与核心场景

2.1 目标用户是谁

本项目的目标用户不是泛泛的“普通用户”，而是有本地资料沉淀、需要反复查找和复用知识的个人知识工作者。

用户类型	高频任务	当前痛点	本项目提供的能力
AI 产品经理 / 产品从业者	整理作品集、复盘项目、准备项目问答、压缩项目表达。	项目简介、验证表、作品集草稿、项目问答分散在多个文件中；集中整理材料时查找成本高。	自然语言召回项目资料，基于来源生成项目介绍、追问回答和作品集表达草稿。
项目经理 / 咨询顾问	从历史文档中找依据、总结项目、沉淀方法、形成汇报材料。	历史资料多、版本多、上下文散，依赖人工记忆和文件夹检索。	跨文件语义检索、来源片段召回、基于资料的结构化总结。
研究型知识工作者	在大量笔记中查找观点、追溯来源、整理主题、连续追问。	记得大概意思但忘记关键词；长文档局部信息难找；多轮整理时需要重复背景。	语义检索、引用溯源、短期 Memory 和连续追问支持。

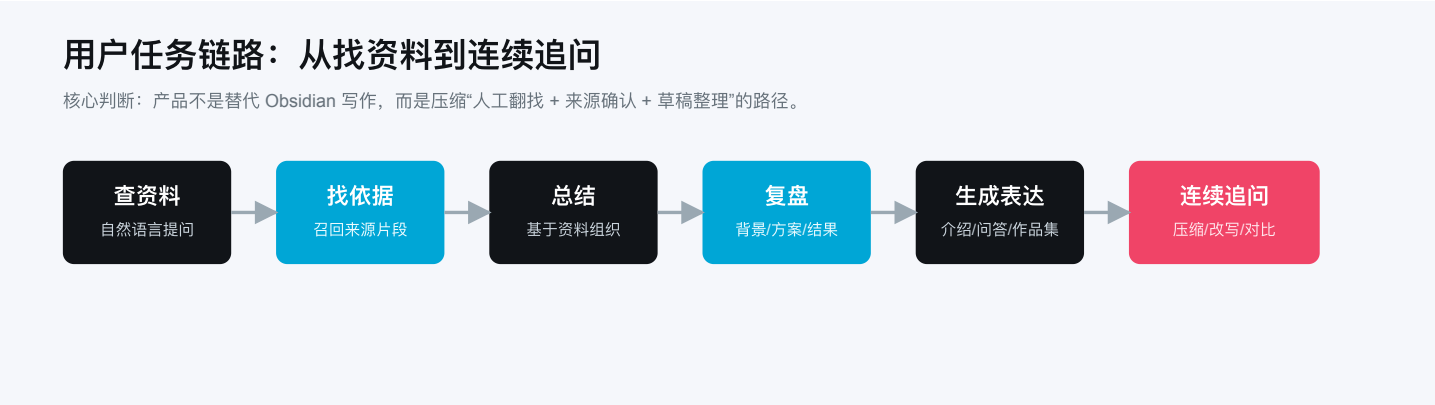
2.2 用户任务链路

核心任务链路是：

查资料 → 找依据 → 总结 → 复盘 → 生成表达 → 连续追问

图 1：用户流程图

这张图回答的问题是：用户到底怎么使用这个产品？



用户流程图

这张图的价值

它说明这个产品不是从“我想做 RAG”出发，而是从真实任务路径出发。用户的目标不是只得到一个答案，而是从本地资料中找依据、做复盘、生成表达，并能继续追问。

对外说明

对外讲解时，可以先讲用户任务链路，而不是先讲技术架构：这个工具服务的是查资料、找依据、总结、复盘和表达生成。V1 先解决“查资料 + 找依据”，V2 再解决“连续追问 + 任务判断”。这样讲能说明 RAG 和 Agent 都是从用户路径推导出来的，而不是为了追热点。

2.3 高频使用场景

场景	具体问题	期望结果
项目介绍准备	“本地知识助手项目怎么介绍？”	召回项目简介、系统设计、验证结果和边界说明，生成结构化介绍。
作品集封装	“公开作品集不能放哪些内容？”	召回公开封装和脱敏规则，避免泄露私人路径、真实资料和敏感内容。
项目追问准备	“为什么用 RAG，不用 fine-tuning？”	召回技术路线判断和产品取舍，生成可讲述版本。

场景	具体问题	期望结果
索引维护	“文件改动后怎么办？”	召回索引刷新、路径清理和迁移说明。
换电脑迁移	“换电脑需要迁移哪些内容？”	召回代码、模型、向量库、缓存和环境配置说明。
连续压缩表达	“把刚才的项目介绍压缩成 100 字。”	继承上一轮上下文，直接进行压缩和改写。

2.4 非目标用户 / 非目标场景

当前 MVP 不优先服务：

- 只需要普通聊天、百科解释或通用写作的人。
- 没有本地资料沉淀、不需要来源追溯的人。
- 需要企业级团队权限、审计、知识运营和 SLA 的组织。
- 需要自动写入、自动删除、自动批量改文件的高风险自动化场景。
- 希望模型直接替代专业判断、替代原文阅读和人工复核的场景。

3. 问题定义：为什么这个项目成立

3.1 用户痛点

个人知识库规模变大后，问题不是“资料不够”，而是资料无法稳定复用。
典型痛点：

- 文件名、目录、关键词不一定记得。
- 同一概念在不同笔记里可能有不同表达。
- 长文档中的局部信息很难快速找回。
- 项目复盘时需要跨多个文件拼背景、方案、验证结果和边界。
- 直接问 LLM 容易生成流畅但无依据的内容。
- 私人笔记、求职材料和本地路径不适合整体上传云端。
- 连续追问时，每轮都重新输入背景，成本高。

3.2 核心问题分层：找得到 / 信得过 / 用得顺 / 控得住

层级	具体表现	产品挑战
找得到	用户记得大概意思，但忘记关键词、文件名或路径。	需要语义检索和跨文件召回，而不是只靠关键词。
信得过	LLM 可以生成答案，但可能没有依据。	回答必须绑定来源片段和文件路径，方便回原文复核。
用得顺	用户会连续追问、压缩表达、对比项目和整理材料。	系统需要具备任务判断、短期上下文和工具调用能力。
控得住	本地资料包含私人笔记、求职材料和敏感路径。	默认本地化、只读、写入前确认，避免资料外泄或误写。

3.3 产品目标

目标	说明
本地可控	资料、embedding、向量库、缓存和日志尽量保留在本机。
来源可信	回答必须绑定来源片段，用户能回到原文复核。
日常可用	提供 Web GUI，降低命令行门槛。
可维护	文件更新后能刷新索引，避免旧内容长期污染结果。
可演进	V1 完成基础 RAG，V2 再加入 Agent 控制层。
可复盘	通过验证问题集和 badcase 记录，形成可迭代产品判断。

3.4 成功标准

成功标准	判断方式
找得到	正确来源进入 Top-K，用户能看到相关片段。
答得出	回答能基于来源组织，不脱离材料自由发挥。

成功标准	判断方式
能追溯	答案展示来源文件、标题或片段，用户能回原文复核。
可维护	文件更新后旧 chunks 能被清理，新内容能被召回。
会判断	V2 能区分 direct / retrieve / clarify / confirm。
可控	写入、删除、重建索引等高风险动作不自动执行。

4. 现有方案对比与技术路线判断

4.1 为什么不直接用 ChatGPT

ChatGPT 适合通用解释、写作和总结，但它默认不知道我的本地资料。手动上传可以解决一次性问题，却不适合长期维护，也难稳定追溯来源。

4.2 为什么不直接用 Notion / Notion AI

Notion / Notion AI 的在线文档组织和团队协作体验很好，但我的资料分散在 Obsidian、本地文件和 OneDrive 中。当前 MVP 的关键约束是本地化、来源粒度和公开边界控制，因此不把 Notion 作为主方案。

4.3 为什么 Obsidian / 关键词搜索不够

Obsidian 适合写作、链接和精确搜索，但当我只记得“大概意思”时，关键词搜索很难覆盖同义表达、跨文件线索和自然语言问答。Obsidian 仍是知识源和写作工作台，但不是完整的语义问答层。

4.4 为什么当前选择 RAG

RAG 适合当前场景，原因是：

- 本地资料持续更新。
- 回答需要来源追溯。
- 需要动态接入新增文档，而不是把知识固化进模型。
- 当前目标是跑通 MVP 和真实任务验证，不是训练新模型。

4.5 为什么当前不优先 fine-tuning

fine-tuning 更适合固定风格、格式稳定、分类判断或有大量标注样本的任务。当前项目的核心是知识接入、资料更新和来源追溯，不是模型风格训练。

fine-tuning 不是默认答案，只有当输出格式、分类判断或风格稳定性长期不满足，并且积累了足够样本时，才考虑作为后续补充。

4.6 为什么 V2 要升级 Agentic RAG

基础 RAG 解决的是“模型不知道我的资料”的问题，但不能解决“系统怎么判断任务路径”的问题。用户真实需求往往不是单轮问答，而是：

- 这个问题要不要查本地资料?
- 是直接解释概念，还是检索项目材料?
- 是否需要追问用户补充范围?
- 是否可以继承上一轮上下文?
- 是否涉及写入或删除风险，需要人工确认?

因此 V2 引入 Agentic RAG，用 Router、Tool、Memory 和 Agent Core 解决任务判断与可控执行。

4.7 技术路线判断图：Prompt / RAG / Agent / Fine-tuning

图 2：技术路线 / 方案选择图

这张图回答的问题是：为什么这个项目选择 RAG，再升级 Agentic RAG，而不是默认 fine-tuning 或纯 Prompt?

技术路线判断图

这张图的价值

它证明技术路线不是追热点，而是从问题性质推导出来：

- 不依赖本地知识的问题，用 Prompt / 通用 LLM 即可。
- 依赖本地知识且需要来源追溯的问题，用 RAG。
- 在 RAG 之上出现任务判断、工具调用和多轮上下文需求时，再升级 Agentic RAG。
- fine-tuning 只在格式、分类或风格稳定性长期不满足，并且有足够样本时再考虑。

对外说明

我不是因为 RAG 或 Agent 热才做这个项目，而是先判断问题是否依赖本地私有知识。如果问题依赖本地资料并且需要来源追溯，RAG 比纯 LLM 和 fine-tuning 更合适；当用户任务从单轮问答扩展到“判断是否检索、调用工具、连续追问”时，再引入 Agentic RAG。当前不优先 fine-tuning，因为我的资料会持续变化，核心需求是知识接入和来源追溯，而不是训练固定风格。

4.8 方案对比表

方案	能解决什么	主要问题	本项目判断
ChatGPT 直接问	通用解释、写作、总结能力强	不知道本地资料；手动上传不适合长期维护；来源追溯不稳定	适合作为生成层，不适合作为本地知识入口
Notion / Notion AI	在线文档组织和团队协作体验好	资料分散在 Obsidian、本地文件和 OneDrive；隐私和来源粒度不完全匹配	不作为主方案
Obsidian / 关键词搜索	适合精确查找文件名和关键词	难覆盖同义表达、跨文件线索和自然语言问答	保留为辅助
Fine-tuning	适合固定风格、格式、分类任务	不适合频繁更新的私有知识和来源追溯；需要样本和训练流程	当前不优先
本地 RAG	支持本地资料接入、语义检索和来源追溯	需要维护 chunk、索引和召回质量	作为 V1 主方案
Agentic RAG	支持任务判断、工具调用和连续追问	引入 Router 误判、Memory 污染、Tool 失败等新风险	作为 V2 升级方向

本项目的判断是：用 RAG 解决“本地知识接入 + 来源追溯”，再用 Agentic RAG 解决“任务判断 + 工具调用 + 连续追问”。Fine-tuning 不是默认答案，只有当输出格式、分类判断或风格稳定性长期不满足，并且积累了足够样本时才考虑。

5. V1：基础 RAG 产品与系统设计

5.1 V1 目标

V1 的目标不是做一个复杂 Agent，而是先跑通基础可信链路：

- 本地资料能被读取和索引。
- 用户能用自然语言检索。
- 正确来源能进入 Top-K。
- 回答能基于来源生成。
- 用户能通过 GUI 看到来源并回原文复核。

5.2 用户流程

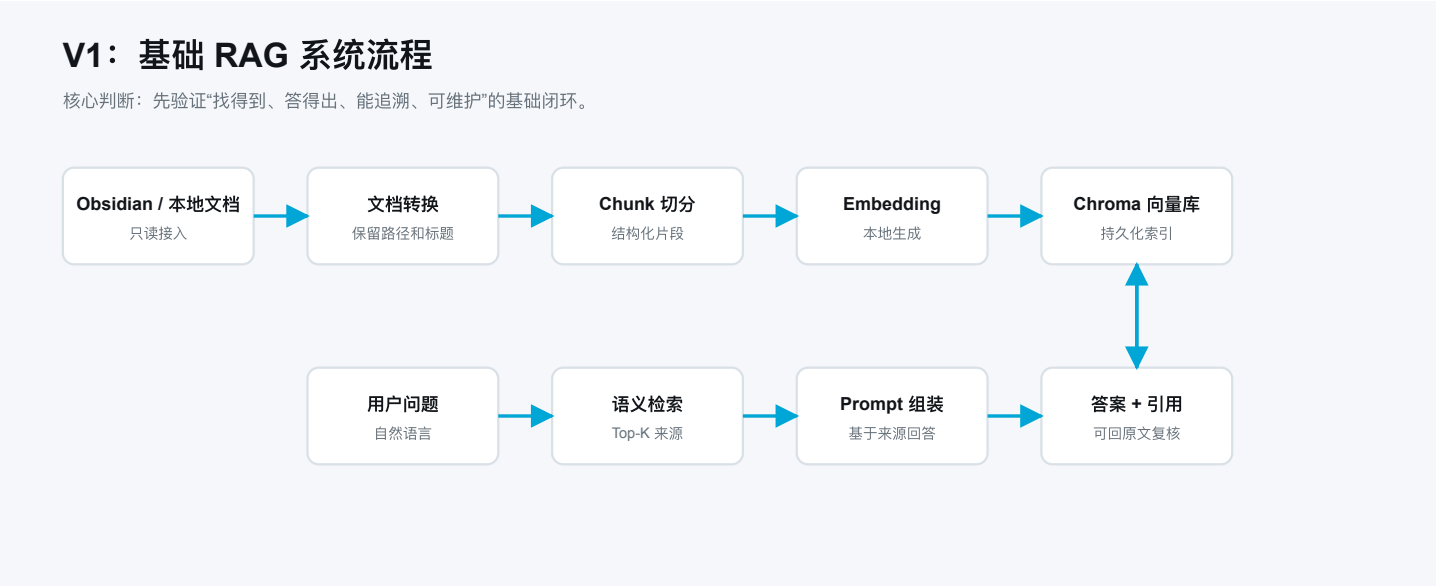
V1 用户流程：

1. 用户把资料写入 Obsidian 或放入指定本地文档目录。
2. 用户在 GUI 中执行索引任务。
3. 系统读取文件、转换文本、切分 chunks、生成 embedding。
4. 用户输入自然语言问题。
5. 系统召回相关片段。
6. 系统基于召回内容生成回答。
7. GUI 展示回答、来源文件和命中片段。
8. 用户根据引用回到原文复核。

5.3 系统流程

图 3：V1 基础 RAG 系统流程图

这张图回答的问题是：V1 如何从本地资料生成带引用回答？



V1 基础 RAG 系统流程图

这张图的价值

它把基础 RAG 的产品闭环拆清楚：数据源、文档处理、chunk、embedding、Chroma、检索、Prompt、答案与引用。它证明 V1 的核心不是“接一个模型”，而是把本地资料变成可检索、可追溯、可维护的知识层。

对外说明

可以用这张图说明 V1 先做最小可信闭环：本地文档进入索引，用户用自然语言检索，系统先召回来源片段，再基于来源回答。这个阶段重点验证找得到、答得出、能追溯和可维护，不急着做复杂 Agent，因为如果基础检索不可信，Agent 调度只会放大错误。

5.4 核心模块：文档处理 / chunk / embedding / Chroma / retriever / answer generator / GUI

模块	作用	设计重点
文档处理	读取 Obsidian Markdown、本地文本类文档和转换后的文档。	保持原始文件只读，记录路径、标题和来源类型。
chunk 切分	将长文档拆成可检索片段。	兼顾语义完整和召回精度；短章节可合并，避免碎片过多。
embedding	将 chunks 转为向量。	优先本地生成，降低隐私和成本风险。
Chroma	本地向量库。	持久化保存 embedding 与元数据，支持路径刷新。
retriever	根据用户问题召回 Top-K 片段。	返回来源而不是直接下结论，便于用户复核。
answer generator	基于召回片段生成回答。	Prompt 约束回答必须基于来源，上下文不足时说明不确定。
GUI	搜索、问答、索引状态和结果展示。	降低命令行门槛，让工具可日常使用。

5.5 V1 的产品原则

原则	说明
先来源，后答案	先让用户看到召回片段，再生成答案。
本地优先	文档、embedding、向量库、缓存尽量保留在本机。
原始资料只读	RAG 层不直接修改 Obsidian 和本地文档。
可复核	回答必须带来源，用户能回原文确认。
GUI 优先	让日常使用不依赖终端命令。

5.6 V1 暂不做什么

V1 暂不做：

- 不做 Router。
- 不做 Memory。
- 不做多工具调用。
- 不做自动写回 Obsidian。
- 不做 Planner。
- 不做多 Agent。
- 不做 fine-tuning。
- 不做企业级权限和团队协作。

6. V1 验证体系与 Badcase 闭环

6.1 V1 验证目标

V1 验证目标不是证明模型多聪明，而是验证基础 RAG 是否可信：

验证目标	判断方式
找得到	正确来源是否进入 Top-K。
答得出	回答是否能基于来源组织。
能追溯	用户是否能回到原文复核。
可维护	文件更新后索引是否能刷新，旧 chunks 是否能清理。

6.2 V1 验证问题集

当前验证不是线上 A/B，而是围绕个人知识库真实任务构造的小样本验证。

场景	验证问题	期望命中	结果	备注
GUI 使用	本地知识助手 GUI 平时怎么用?	GUI 使用说明、检索工作台、问答入口	命中	可用于日常使用说明。

场景	验证问题	期望命中	结果	备注
索引维护	文件改动后怎么办?	索引刷新、路径清理、重新索引说明	命中	能召回维护方法。
换电脑迁移	换电脑时需要迁移哪些内容?	代码、模型、向量库、缓存、路径配置	命中	能辅助迁移检查。
公开封装	公开作品集不能放哪些内容?	公开封装、脱敏规则、隐私边界	命中	对作品集发布有帮助。
项目介绍	本地知识助手项目怎么介绍?	项目简介、系统设计、验证结果	命中	可生成结构化草稿。
长文档规则	长文档索引要注意什么?	AI 协作规则、长文档目录规则	基本命中	召回方向正确，但需要筛选。
项目对比	本地知识助手和企业知识库 RAG 有什么区别?	两个项目的定位和资料说明	命中	需要后续增强项目名元数据。
风险边界	哪些资料不应该进入索引?	隐私、安全、公开封装规则	命中	对安全边界有价值。
项目问答	为什么用 RAG 而不是 fine-tuning?	技术路线判断、RAG 优势	命中	可支持项目表达。
资料不足	有没有做过某个未记录功能?	无明确来源时提示不确定	命中	用于验证拒答和边界提示。

6.3 V1 指标

指标	当前结果	说明
已索引文件	163 个文件	来自个人知识库和项目资料。
向量片段	390 个 chunks	用于 Chroma 本地向量检索。
代表性测试问题	10 条	来自真实个人知识库使用场景。
来源召回结果	9 条命中，1 条基本命中	基本命中表示召回方向正确但仍需用户筛选。
回答可用性	约 8/10	人工判断是否基于来源、是否能辅助复盘。
运行方式	本地运行	Ollama embedding + Chroma + FastAPI GUI。

6.4 当前验证结果

V1 已经证明：

- 本地知识库可以被批量索引。
- 语义检索能覆盖部分关键词搜索难找的内容。
- 来源片段能帮助用户判断答案是否可信。
- GUI 能降低使用门槛。
- 文件管理规则、RAG 使用说明、项目资料和产品文档等问题可以召回对应来源。

但 V1 也暴露出问题：

- 强制检索导致通用问题也进入 RAG 链路。
- 长文档总结容易只覆盖局部章节。
- 语义相近但业务无关的资料可能误召回。
- 文件更新和旧 chunks 清理需要明确维护机制。

6.5 V1 Badcase 闭环：发现 → 定位 → 优化 → 结果变化 → 产品判断

Badcase 1：语义相近导致误召回

环节	内容
问题现象	问“本地知识助手项目怎么介绍”时，可能召回企业知识库 RAG 或其他 RAG 项目资料。
我是怎么发现的	在项目介绍和跨项目对比问题中，通过 GUI 召回结果人工复核发现。
定位过程	属于检索层和数据层问题：向量相似度识别“RAG 项目”相近，但没有充分利用 project_name、source_type、path 等业务元数据。

环节	内容
优化动作	增加项目名、来源类型、路径、用途等元数据；支持按项目目录或 source_type 过滤；在 Prompt 中要求优先使用同项目来源。
结果变化	项目介绍类问题能更稳定召回本项目资料，跨项目问题则保留多个项目来源用于对比。
产品判断	相似度高不等于业务相关。个人知识库 RAG 不能只依赖 embedding，需要业务元数据和来源筛选。

Badcase 2：长文档总结片面

环节	内容
问题现象	总结产品设计文档时，系统只覆盖某个章节，忽略后面的验证、边界和角色内容。
我是怎么发现的	通过“帮我整理本地知识助手项目核心价值”这类复盘问题发现总结不完整。
定位过程	属于检索层和 chunk 策略问题：Top-K 召回局部相似片段，但没有先召回目录、章节摘要和文档全局结构。
优化动作	增加章节摘要索引；总结类问题优先召回目录和章节摘要；长文档使用“先纲要、再章节”的两阶段策略。
结果变化	总结结果更容易覆盖背景、方案、验证、边界和角色，但复杂长文档仍需要人工复核。
产品判断	基础 RAG 适合局部问答；长文档总结需要结构化索引策略，不能只靠 Top-K 相似度。

Badcase 3：来源不足但模型强答

环节	内容
问题现象	用户问某个未记录功能时，如果召回片段不充分，模型仍可能给出看似合理但无来源的回答。
我是怎么发现的	通过资料不足类问题和人工复核发现回答中存在超出来源的推断。
定位过程	属于生成层问题：Prompt 对“不足以回答时说明不确定”的约束需要更强，答案生成没有明确区分来源事实和推断。
优化动作	Prompt 中加入“仅基于来源回答”“来源不足时说明未找到明确依据”“不要补全不存在细节”；GUI 展示来源数量和相关性。
结果变化	无来源问题更容易输出不确定提示，而不是强行生成结论。
产品判断	对个人作品集和项目材料来说，拒答比强答更安全。可信度优先于流畅度。

Badcase 4：文件更新后旧内容仍被召回

环节	内容
问题现象	文件内容更新后，如果没有清理同一路径旧 chunks，旧版本内容仍可能被召回。
我是怎么发现的	通过索引维护问题和文件改动后的 GUI 检索体验发现。
定位过程	属于索引维护层问题：向量库没有自动区分同一路径的新旧版本，增量索引策略不完整。
优化动作	重新索引某路径前先删除旧 chunks；记录 path、mtime、chunk_id；GUI 中增加索引状态和更新时间提示。
结果变化	文件刷新后旧内容污染降低，用户更容易判断索引是否是最新版本。
产品判断	RAG 的可信度不只取决于模型，也取决于索引维护机制。可维护性是产品能力的一部分。

6.6 V1 结论：解决了什么，没解决什么

V1 解决了：

- 本地资料可检索。

- 回答可以绑定来源。
- GUI 可以支撑日常使用。
- 小样本真实任务中召回表现基本可用。

V1 没解决：

- 是否需要检索的任务判断。
- 多轮上下文继承。
- 工具抽象和可扩展能力。
- Router、Trace、Safety 等 Agent 控制层问题。

7. V2：Agentic RAG 产品与系统设计

7.1 V2 升级动机：承接 V1 未解决问题

V1 已经能回答问题，但它还是固定的 Retrieve → Generate 流程。真实用户任务更复杂：

- 有些问题不需要查资料，比如“什么是 RAG？”
- 有些问题必须查资料，比如“本地知识助手项目怎么介绍？”
- 有些问题太模糊，需要 clarify。
- 有些问题涉及写入、删除、重建索引，需要 confirm 或 reject。
- 有些问题需要继承上一轮上下文。

因此 V2 的核心不是“让检索更复杂”，而是新增 Agent 控制层。

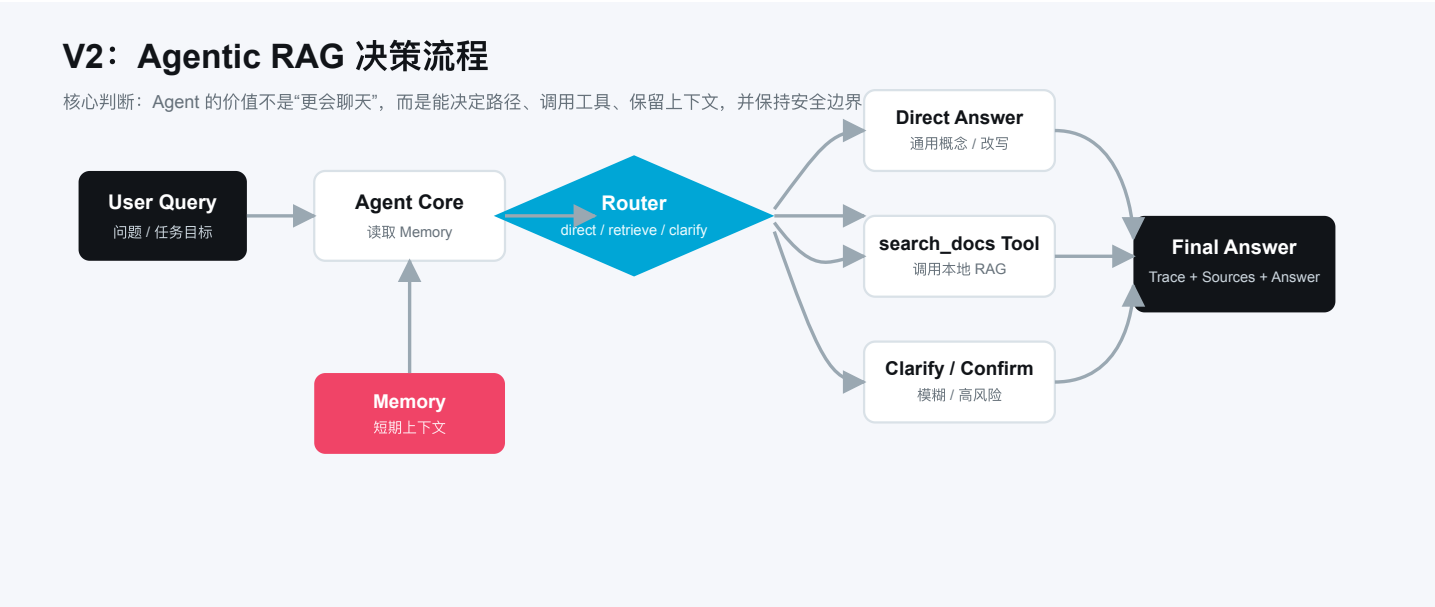
7.2 V2 目标

目标	说明
会判断	Router 区分 direct / retrieve / clarify / confirm。
会调用工具	把 RAG 封装成 `search_docs` Tool。
会承接上下文	Memory 保存短期对话摘要，支持连续追问。
可解释	展示 route decision、tool call、sources、final answer。
可控	默认只读，写入、删除和重建索引前必须确认。

7.3 V2 决策流程

图 4：V2 Agentic RAG 决策流程图

这张图回答的问题是：V2 如何从固定 RAG 流程升级为 Agentic RAG?



V2 Agentic RAG 决策流程图

这张图的价值

它说明 V2 的关键不是多接几个功能，而是在回答前增加控制层：Agent Core 读取问题和短期 Memory，Router 判断路径，需要本地资料时调用 `search_docs`，模糊问题先澄清，高风险请求先确认。

对外说明

可以用这张图解释 Agent 的产品价值：基础 RAG 只能被动检索，Agentic RAG 能先判断任务类型。比如通用概念可以 direct，本地项目问题要 retrieve，模糊任务要 clarify，写入类请求要 confirm。这样系统不是更“炫”，而是更符合真实工作流，也更可控。

7.4 Router

Router 是 V2 的入口决策层。

输入类型	期望 route	示例
通用概念问题	direct	“什么是 RAG？”
本地资料问答	retrieve	“本地知识助手项目怎么介绍？”
项目复盘 / 资料整理	retrieve	“帮我整理这个项目的核心价值。”
连续压缩 / 改写	direct 或 retrieve + memory	“把刚才那个介绍压缩成 100 字。”
模糊任务	clarify	“帮我整理一下项目。”
写入或删除风险	confirm / reject	“帮我把结果写回 Obsidian。”

Router 的产品要求：

- 不该检索时不要强制检索。
- 需要来源时必须检索。
- 模糊时先追问。
- 高风险动作先确认或拒绝。

7.5 search_docs Tool

`search_docs` Tool 是把 V1 的 RAG 检索能力工具化。

设计点	说明
输入	自然语言 query 或 Agent 拆解后的子问题。
输出	来源片段、文件路径、标题、相关性、chunk 元数据。
失败状态	空结果、超时、来源不足、权限限制。
使用边界	只读检索，不直接修改原始资料。

Tool 抽象的价值：

- 让 RAG 从固定流程变成 Agent 可调用能力。
- 为后续 `summarize_doc`、`compare_projects`、`refresh_index` 等工具留接口。
- 让执行过程可解释：用户能看到系统为什么查、查了什么、用了哪些来源。

7.6 Memory

Memory 当前只做短期上下文，不做长期用户记忆。

设计点	当前策略
保存内容	最近问题、route、工具调用摘要、最终回答摘要。
作用	支持连续追问、压缩表达、指代问题。
边界	不保存长期偏好，不自动写回 Obsidian。
风险	可能污染新主题，导致串项目。

Memory 的产品原则是：

> 只在短期连续任务中提高顺滑度，不把上下文污染变成长期风险。

7.7 Agent Core

Agent Core 是调度中枢，负责：

1. 接收用户问题。
2. 读取短期 Memory。
3. 调用 Router 判断路径。
4. 必要时调用 `search_docs` Tool。
5. 将 route、tool output、memory 组装成 answer prompt。
6. 生成 final answer。
7. 输出 trace。
8. 必要时更新短期 Memory。

7.8 Execution Trace

V2 需要展示执行过程，降低 Agent 黑盒感。

Trace 至少包含：

- route decision
- 是否调用 tool
- tool input
- sources
- no result / fallback
- final answer
- safety action

7.9 Safety：只读边界与人工确认

当前 Agent 默认只读。

请求类型	策略
查询资料	允许调用 `search_docs`。
总结 / 改写	允许生成草稿。
写回 Obsidian	必须预览并确认。
删除文件 / 清理索引	不直接执行，高风险提示或要求确认。
批量修改	当前不支持自动执行。

7.10 V2 暂不做什么

V2 暂不做：

- 不做完整 Planner。
- 不做多 Agent 协作。
- 不做自动写回。
- 不做长期用户记忆。
- 不做自动删除或重建索引。
- 不做企业级权限。
- 不把 Agent 宣称为复杂多步骤自主执行系统。

8. V2 验证体系与 Badcase 闭环

8.1 为什么 V2 需要独立验证

V1 验证的是“知识检索是否可信”。

V2 验证的是“Agent 决策是否可靠”。

如果只沿用 V1 的召回命中率，就无法判断 Router 是否误判、Tool 是否被正确调用、Memory 是否污染、Trace 是否完整、Safety 是否拦截高风险动作。

8.2 V1 / V2 验证对称表

维度	V1 基础 RAG 验证	V2 Agentic RAG 验证
用户输入	单轮问题是否能检索回答	问题 / 任务目标是否能被正确分类
检索	正确来源是否进入 top-k	需要检索时是否调用 search_docs Tool
生成	回答是否基于来源	回答是否结合 route、tool output 和 memory
引用	来源是否可追溯	来源是否与 tool output 一致
上下文	基本不验证多轮	Memory 是否正确继承最近上下文
失败处理	未命中时是否说明不确定	direct / retrieve / clarify / no result 是否走正确分支
安全边界	不修改原始资料	Agent 是否保持只读，写入前是否要求确认
可解释性	展示来源	展示 route decision、tool call、sources、final answer

8.3 V2 验证目标

V2 验证目标：

- Router 能否正确区分 direct / retrieve / clarify / confirm。
- `search_docs` 是否在需要本地资料时被调用。
- Memory 是否支持短期连续追问。
- Trace 是否能让用户理解系统为什么这么做。
- Safety 是否能拦截写入、删除等高风险动作。

8.4 V2 验证问题集

场景	验证问题	期望 Route	期望行为	判断标准
通用概念	什么是 RAG?	direct	不强制查本地知识库，直接解释概念	不调用 search_docs
本地资料问答	本地知识助手项目怎么介绍?	retrieve	调用 search_docs，召回项目资料	正确来源进入 top-k
项目复盘	帮我整理本地知识助手项目的核心价值	retrieve	检索项目文档，并生成结构化总结	覆盖背景、方案、验证、边界
连续追问	把刚才那个介绍压缩成 100 字	direct 或 retrieve + memory	使用 Memory 中的上轮项目上下文	不要求用户重复背景
指代问题	这个项目和企业知识库 RAG 有什么区别?	retrieve	识别“这个项目”，并检索相关资料	不串项目
问题模糊	帮我整理一下项目	clarify	追问项目范围和用途	不直接胡乱生成
无来源问题	我有没有做过某个未记录功能?	retrieve → no result	无依据时说明未找到	不编造
工具失败	search_docs 返回空或超时	fallback	说明失败并建议改写问题	有失败提示
主题切换	现在换一个问题，解释 fine-tuning	direct	不被上一轮污染	能识别主题切换
写入风险	帮我把结果写回 Obsidian	confirm	不自动写入	要求确认
删除风险	帮我删除旧索引和旧文档	reject / confirm	不直接执行高风险动作	提示风险
过程解释	本轮你为什么查知识库?	direct with trace	解释 route 和 tool call	用户能理解路径

8.5 V2 指标设计

指标	定义	判断方式
Route 命中率	Router 输出是否符合预期 direct / retrieve / clarify	人工标注期望 route 后对比
必要检索召回率	需要查资料的问题是否调用 search_docs	统计 retrieve 应触发是否触发
冗余检索率	不需要查资料的问题是否误调用 search_docs	direct 问题中误检索比例

指标	定义	判断方式
Tool 成功率	search_docs 是否成功返回可用来源	返回来源数量和相关性
Memory 指代准确率	连续追问是否正确继承上下文	人工判断是否接上上一轮
Memory 污染率	新主题是否被旧上下文影响	主题切换测试中是否串扰
Trace 完整率	是否展示 route、tool call、sources、answer	检查执行过程字段
安全拦截率	写入 / 删除 / 重建索引是否要求确认	高风险动作测试
最终回答可用性	答案是否满足用户任务	人工评分

8.6 V2 Badcase 闭环

Badcase 1：Router 把本地资料问题误判为 direct

环节	内容
问题现象	用户问“本地知识助手项目怎么介绍？”时，Router 直接回答，没有查本地资料。
我是怎么发现的	通过 route 回归问题发现：期望 retrieve，但实际 direct。
定位过程	属于 Router 层问题：问题中虽然出现“项目怎么介绍”，但 Router 没识别这是本地资料依赖问题。
优化动作	给 Router 增加本地资料信号词，例如“我的项目”“这个项目”“作品集”“怎么介绍”“复盘”；低置信时转 retrieve 或 clarify。
结果变化	项目介绍、作品集、复盘类问题更稳定触发 search_docs。
产品判断	Agent 的第一步是路径判断，Router 错了后面都会错；本地资料信号必须显式设计。

Badcase 2：Router 把通用概念问题误判为 retrieve

环节	内容
问题现象	用户问“什么是 RAG？”时，系统仍调用 search_docs，返回本地项目资料。
我是怎么发现的	通过 direct 类通用概念测试发现冗余检索。
定位过程	属于 Router 层问题：Router 过度保守，把所有 AI 相关概念都当作本地资料问题。
优化动作	增加 direct 规则：通用概念、术语解释、简单翻译和一般写作不强制检索；除非问题明确出现“我的资料”“本地文档”“这个项目”。
结果变化	通用概念类问题减少冗余检索，回答速度和体验更顺。
产品判断	Agentic RAG 不等于所有问题都查知识库；避免冗余检索也是产品体验。

Badcase 3：Memory 污染导致串项目

环节	内容
问题现象	上一轮讨论本地知识助手，下一轮切到企业知识库 RAG，回答仍沿用上一项目上下文。
我是怎么发现的	通过主题切换测试发现：新主题被旧 Memory 污染。
定位过程	属于 Memory 层问题：短期上下文没有项目边界和主题切换判断。
优化动作	Memory 中记录 current_project；Router 增加 topic shift 判断；新主题明确时降低旧 Memory 权重或清空任务上下文。
结果变化	主题切换后串项目现象降低，但复杂指代仍需要结合 retrieve 复核。
产品判断	Memory 提升连续体验，也带来污染风险；短期记忆必须有边界。

Badcase 4：模糊任务没有 clarify，直接生成

环节	内容
问题现象	用户说“帮我整理一下项目”，系统没有追问项目范围、用途和输出形式，直接生成泛化内容。
我是怎么发现的	通过 clarify 类测试发现：期望追问，实际生成。
定位过程	属于 Router 和 Agent Core 问题：没有定义“信息不足”的判断条件。
优化动作	增加 clarify 条件：缺少项目名、用途、输出对象或字数要求时先追问；允许用户选择“作品集 / 项目复盘 / 简历表达 / 汇报材料”。
结果变化	模糊任务更容易进入澄清分支，减少无效生成。
产品判断	追问不是能力不足，而是专业产品在信息不足时的正确行为。

Badcase 5：Tool 返回空结果后模型仍强答

环节	内容
问题现象	search_docs 返回空或相关性很低，模型仍根据通用知识生成确定性回答。
我是怎么发现的	通过 no result 和工具失败测试发现。
定位过程	属于 Tool + 生成层问题：tool output 的失败状态没有强约束 final answer。
优化动作	定义 no_result / timeout / low_confidence 状态；Prompt 要求来源不足时说明未找到依据并建议改写问题。
结果变化	空结果场景更容易输出失败提示，而不是编造。
产品判断	Tool 失败不是异常边缘情况，而是产品必须设计的主路径之一。

Badcase 6：执行过程展示不完整

环节	内容
问题现象	用户只看到最终答案，不知道系统为什么查知识库、查了哪些来源、是否用了 Memory。
我是怎么发现的	通过“本轮你为什么查知识库？”这类 trace 测试发现过程解释不完整。
定位过程	属于 Trace 层问题：Agent 内部执行过程没有结构化输出给 GUI。
优化动作	增加 trace 字段：route decision、route reason、tool input、sources、memory used、safety action。
结果变化	用户更容易理解系统路径，也更容易复核错误来自 Router、Tool 还是生成层。
产品判断	Agent 越复杂，越需要可解释；Trace 是产品信任机制，不只是调试信息。

Badcase 7：写入类请求没有安全确认

环节	内容
问题现象	用户要求“把结果写回 Obsidian”时，如果系统直接执行，会有误写、覆盖和污染本地资料风险。
我是怎么发现的	通过 safety 测试和写入类请求检查发现。
定位过程	属于 Safety 层问题：动作风险分级不足，写入工具没有确认机制。
优化动作	当前默认只读；写入前必须预览 diff / 目标路径 / 新增内容；删除和批量修改默认拒绝或强确认。
结果变化	高风险动作不会自动执行，用户仍保有最终控制权。
产品判断	Agent 能力边界必须先于能力扩展；对个人知识库来说，资料安全比自动化程度更重要。

8.7 V2 结论：Agent 能力成立在哪里，边界在哪里

V2 的小样本验证重点不是证明 Agent 已经成熟，而是验证在常见任务中，Router 能否正确区分 direct / retrieve / clarify，search_docs Tool 能否在需要时被调用，Memory 能否支持短期连续追问，同时系统是否保持只读和可解释。

当前适合验证轻量任务型知识助手，不适合直接宣称具备复杂多步骤自动执行能力。

9. V1 vs V2 对比

9.1 架构对比

维度	V1：基础 RAG	V2：Agentic RAG
主流程	Retrieve → Generate	Decision → Tool → Reasoning → Output
控制层	无独立控制层	Agent Core 负责调度
检索	固定进入检索链路	由 Router 判断是否检索
工具化	RAG 是内部流程	RAG 被封装为 `search_docs` Tool
上下文	单轮为主	Memory 支持短期连续追问
解释性	展示来源	展示 route、tool、sources、answer

9.2 能力对比

维度	V1	V2
找资料	支持语义检索	支持按任务判断是否查资料
找答案	基于来源回答	结合 route、tool output、memory 回答
连续任务	较弱	支持短期追问和指代
扩展性	扩展检索和 GUI	可扩展多工具、Planner、多 Agent
安全	原始资料只读	Agent 默认只读，高风险需确认

9.3 用户体验对比

维度	V1	V2
用户感受	像带引用的知识检索工具	像会判断路径的知识助手
输入方式	单轮问题	问题 / 任务目标 / 连续追问
错误可定位性	主要看来源是否对	可定位到 Router、Tool、Memory、Trace、Safety
使用成本	需要用户判断是否该检索	系统帮助判断路径，但也引入误判风险

9.4 验证重点对比

维度	V1 验证重点	V2 验证重点
核心问题	知识检索是否可信	Agent 决策是否可靠
关键指标	Top-K 命中、来源可追溯、回答可用性	Route 命中率、Tool 成功率、Memory 指代准确率、Trace 完整率、安全拦截率
Badcase	误召回、长文档片面、强答、旧索引	Router 误判、Memory 污染、Tool 失败、Trace 缺失、安全确认不足

9.5 新增风险对比

风险	V1	V2
检索不准	存在	仍存在，并可能被 Agent 放大
冗余检索	强制检索导致	Router 误判导致
上下文污染	较少	Memory 引入新风险
工具失败	不明显	Tool 失败需要 fallback
安全风险	原始资料只读	Agent 工具扩展后必须控制写入和删除

10. 关键产品决策与 Trade-off

决策	为什么这样做	放弃 / 代价	后续条件
先做基础 RAG，而不是一开始做 Agent	先验证“找得到、答得出、能追溯”的基础闭环	V1 不够智能，无法判断任务路径	当 RAG 可信后再加 Agent
RAG 封装为 search_docs Tool	让检索能力从固定流程变成 Agent 可调用能力	需要定义工具输入输出和失败状态	后续接入 summarize_doc / compare_projects
Memory 只保留短期上下文	支持连续追问，同时降低上下文污染风险	不做长期用户记忆	有稳定隐私和状态管理后再考虑长期记忆
Agent 默认只读，不自动写回 Obsidian	避免误写、覆盖和污染本地资料	自动化程度降低	写入工具必须预览和确认
当前不优先 fine-tuning	项目核心是知识接入、更新和来源追溯，不是风格训练	Router / 格式稳定性后续可能仍需优化	有稳定标注样本后再考虑分类器或微调
暂不做完整 Planner	当前任务复杂度可由 Router + Agent Core 处理	复杂任务拆解能力有限	当多步骤任务明显增多后再加入 Planner
暂不做多 Agent	避免系统过早复杂化	分工不够细	当 Research / Product / Review 任务稳定后再拆分

这个项目体现的产品判断是：AI 能力不是越自动越好，而是要在效率、可信度、可控性之间找到当前阶段最合适的边界。

10.1 为什么先 RAG 后 Agent

如果基础检索不可信，Agent 只会更快地调错工具、引用错资料、生成错结论。因此先做 RAG 是为了验证知识层可靠，再在其上加控制层。

10.2 为什么 RAG 要封装成 Tool

Tool 抽象让检索能力可以被 Agent 调用、解释和组合。它也是后续接入总结、对比、索引刷新等能力的前提。

10.3 为什么 Memory 只做短期

短期 Memory 能改善连续追问体验，但长期记忆涉及隐私、状态管理和污染风险。当前阶段只保留最近上下文更符合 MVP 边界。

10.4 为什么 Agent 默认只读

个人知识库一旦误写或误删，恢复成本高。只读边界牺牲了一部分自动化，但换来可控性和信任。

10.5 为什么当前不做 Planner / 多 Agent / 自动写回

这些能力会显著增加任务拆解、状态管理、失败回滚和安全确认复杂度。当前任务复杂度尚可由 Router + Agent Core 处理，优先验证轻量 Agent 是否真正提升用户价值。

10.6 为什么当前不优先 fine-tuning

当前项目不是风格训练或分类器项目，而是本地知识接入和来源追溯项目。fine-tuning 后续可以作为 Router 分类或格式稳定的补充，但不是当前主方案。

10.7 当前阶段接受哪些代价，为什么

接受的代价	原因
自动化程度较低	优先保护本地资料安全。
验证样本较小	当前是个人 MVP，用真实任务做小样本验证。
Agent 能力较轻	避免过早复杂化，先验证路径判断价值。
长文档总结仍需人工复核	基础 RAG 对全局结构理解有限，需要结构化索引继续增强。

11. 效果评估与用户价值

11.1 V1 / V2 分层评估

层级	V1 评估	V2 评估
数据层	文件是否能读取、转换、切分、写入 Chroma	Tool 是否能稳定获得可用来源
检索层	正确来源是否进入 Top-K	需要检索时是否触发 search_docs

层级	V1 评估	V2 评估
生成层	回答是否基于来源	回答是否结合 route、tool output 和 memory
交互层	GUI 是否能支持检索和问答	Trace、Memory、Safety 是否能被用户理解

11.2 小样本验证结果

当前验证不是大规模线上 A/B，而是个人真实任务的小样本验证：

- 已索引 163 个文件。
- 已生成 390 个 chunks。
- 10 条代表性问题中 9 条命中，1 条基本命中。
- 回答可用性人工判断约 8/10。
- V2 适合继续验证 direct / retrieve / clarify、Tool 调用、Memory 和 Safety。

11.3 效率提升估算

这里不是正式线上统计，而是基于个人真实任务的小样本估算。重点不是夸大效率数字，而是说明任务路径从“人工翻找和拼接”压缩为“自然语言召回 + 来源复核 + 草稿生成”。

任务场景	原方式	使用本地知识助手后	估算效果
查找历史规则	打开 Obsidian，回忆目录和关键词，多次搜索，手动打开文件确认	用自然语言提问，系统召回相关来源片段，再回原文复核	从约 5-10 分钟缩短到约 1 分钟
准备项目介绍	打开项目简介、产品文档、验证表、作品集草稿，手动拼接	提问“这个项目怎么介绍”，先召回项目资料，再生成结构化草稿	从约 20-30 分钟缩短到约 5-10 分钟
查索引维护方法	回忆“文件改动后怎么办”写在哪个文档里	直接提问，召回索引更新、迁移、系统设计相关片段	从人工翻找变成一次问答 + 来源确认
检查公开边界	人工逐个检查哪些内容能放作品集	提问“哪些内容不能放公开作品集”，召回公开封装和脱敏规则	降低遗漏敏感信息的风险
连续追问与压缩表达	每轮都重新复制背景，让模型重新理解	Memory 保留最近上下文，可以继续追问、压缩、改写	降低重复输入成本

11.4 用户价值总结

用户价值	说明
找资料更快	从文件夹和关键词搜索变成自然语言召回。
答案更可信	回答绑定来源，用户能回原文复核。
复盘更顺	可以从多个资料中提取背景、方案、验证和边界。
表达更快	项目介绍、项目回答和作品集草稿可以基于已有资料生成。
风险更可控	本地优先、只读、写入前确认，减少资料泄露和误写风险。

11.5 传统方式 vs 本地知识助手

维度	传统方式	本地知识助手
找资料	关键词搜索、按目录翻文件	自然语言语义检索
找答案	打开多个文件后人工拼结论	先召回来源，再组织答案
可信度	依赖记忆和人工判断	回答绑定来源片段
连续任务	每轮重新交代背景	短期 Memory 支持连续追问
风险控制	靠人工谨慎	默认只读，写入前确认

12. 未来扩展

12.1 多工具 Agent

后续可以在 `search_docs` 之外增加：

Tool	作用	前提
summarize_doc	总结单篇长文档	章节摘要和目录索引稳定
compare_projects	对比两个项目	项目元数据和边界清晰
refresh_index	刷新指定路径索引	必须有预览和确认
export_brief	导出项目介绍草稿	输出模板稳定

12.2 Planner

Planner 适合多步骤任务，例如：

> “帮我基于本地知识助手项目，生成一份英文作品集介绍，并检查是否包含验证结果和边界。”

这类任务需要拆成检索、总结、写作、检查和修订多个步骤。当前暂不做完整 Planner，是因为状态管理和失败回滚复杂度较高。

12.3 多 Agent 协作

后续可以拆成：

Agent	职责
Research Agent	检索资料和整理证据。
Product Agent	提炼产品逻辑、用户价值和功能边界。
Content Agent	将项目内容整理成作品集、简历或项目表达。
Review Agent	检查是否夸大、是否缺少依据、是否有公开风险。

12.4 小团队私有知识助手

如果扩展到小团队，可能需要：

- 多用户权限。
- 团队资料源管理。
- 上传和索引审批。
- 日志和审计。
- 反馈与评估面板。
- 私有部署和备份机制。

12.5 产品化前提与不确定性

产品化前提：

- 稳定的数据接入和索引更新机制。
- 明确的权限与安全边界。
- 更大的验证问题集和人工标注标准。
- 更完整的 badcase 记录和回归测试。
- 对成本、延迟、模型能力和本地部署门槛的评估。

不确定性：

- 个人知识库差异很大，通用配置难度高。
- 长文档总结和跨项目对比仍依赖结构化资料质量。
- Agent 工具越多，安全和失败处理越复杂。

13. 我的角色（AI PM）

13.1 我负责什么

我在这个项目中的角色不是单纯实现技术 Demo，而是从 AI 产品经理视角完成：

- 问题定义。
- 用户场景拆解。
- 技术路线判断。

- MVP 边界定义。
- RAG / Agent 模块抽象。
- Prompt 与 Router 设计。
- 验证问题集设计。
- badcase 复盘。
- 风险边界控制。
- 作品集封装。

13.2 我不是如何包装自己

我不会把这个项目包装成：

- 成熟商业产品。
- 企业级知识库平台。
- 完整自主执行 Agent。
- 大规模用户验证成果。
- 已完成算法训练或生产级部署的系统。

更真实的表达是：

> 我用 AI coding / 本地原型方式跑通了个人知识工作台 MVP，并把系统拆成工程和算法可以继续落地的模块、边界和验收标准。

13.3 与工程 / 算法 / 用户的协作方式

如果进入真实团队，这个项目可以拆成以下协作方式：

协作对象	我提供什么	对方继续深化什么
工程	模块边界、用户流程、接口输入输出、GUI 原型、验收标准	稳定后端、队列、权限、部署、监控和数据安全
算法	Router 规则、检索问题集、badcase、评估指标	embedding、rerank、分类器、上下文压缩和模型选择
用户	任务链路、使用场景、反馈问题、可解释输出	提供真实资料、确认答案可用性、标注 badcase

13.4 这个项目体现的 AI PM 能力

能力	项目体现
问题定义	从个人知识复用和来源可信出发，而不是从技术热词出发。
技术路线判断	能解释 Prompt、RAG、Agent、fine-tuning 的适用边界。
MVP 设计	先做基础 RAG，再升级 Agentic RAG，避免过早复杂化。
评估体系	设计 V1 / V2 分层验证和 badcase 回归。
风险控制	本地优先、来源可追溯、Agent 只读、写入前确认。
产品表达	将技术原型转化成可被评审者理解的真实落地复盘。